Verification of FIR filter

Contents

Introduction

Filters are one of the most basic Building Blocks in DSP (whether it's for an ASIC or to be used on FPGA), They are vital blocks in any system.

Mainly, A Filter is a LTI system that is used as a Signal Conditioning block which main role is to select the range of frequencies that will be either filtered out or allowed to pass through and it has 4 main types:

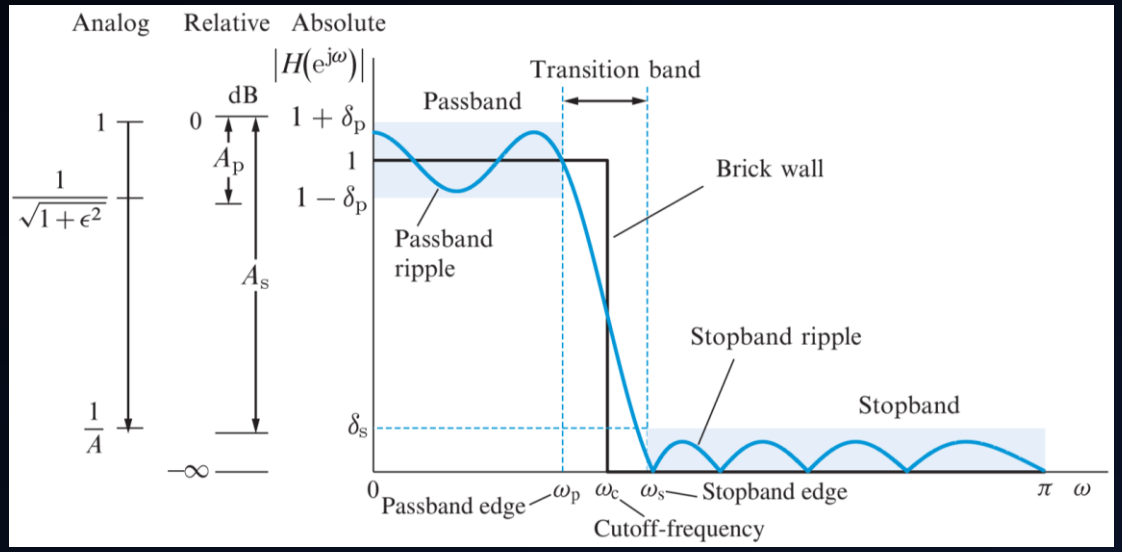
a) Low Pass Filter (LPF): allows frequencies that are below a certain threshold (Fc) to pass while attenuating above frequencies.

b) High Pass Filter (HPF): allows frequencies that are above a certain threshold (Fc) to pass while attenuating below frequencies.

c) Band Pass Filter (BPF): allows frequencies that are between two limits (Fc1 & Fc2) to pass while attenuating other frequencies.

d) Band Reject (Stop) Filter: attenuates frequencies that are between two limits (Fc1 & Fc2) while allowing other frequencies.

- We will implement Lowpass FIR filter:
- FIR filter is an Impulse response that settles at zero after a finite period of time, hence called a Finite Impulse Response.



$$y[n] = \sum_{k=0}^{M} b_k \, x[n-k]$$

- $x[n–k]$: input sampled signal (discrete)
- $y[n]$: The Filtered signal (FIR output)
- bk : Filter Coefficients, which equals the impulse response.

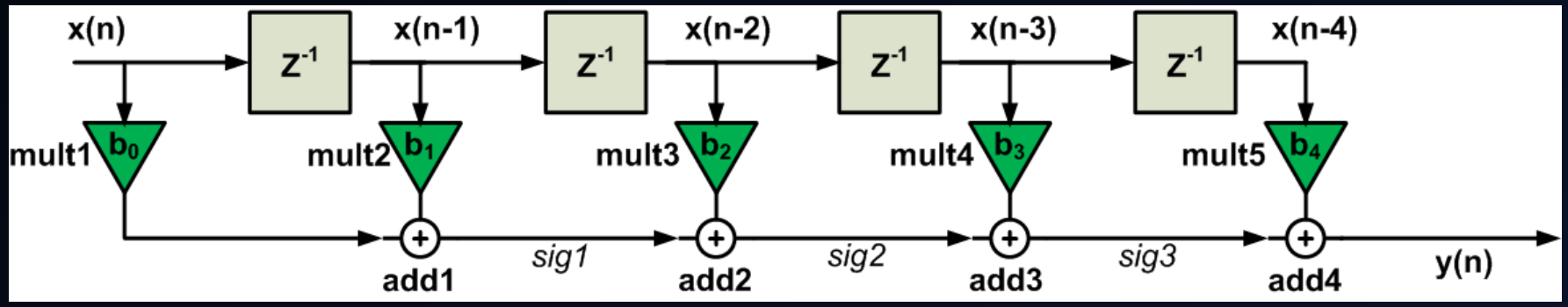
Filter Design

- **FIR Design will go by three main blocks:**

  1) Delay elements: represents the Z-transform block which will clock each samples delay properly.

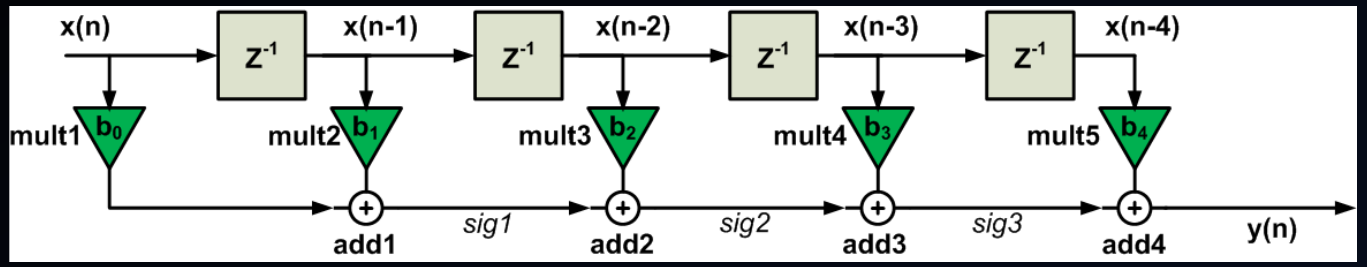
  2) Multipliers: for each of the taps coefficient values (impulse response) to be multiplied by the input samples.

  3) Adders: The products from the multipliers are summed together to produce the filter's output.



- A filter with N^{th} order will have (N) Z-transform blocks and adders, However It'll have (N+1) multiplier taps.
- for example: In the figure above, We have N adders & N circular buffers, (N+1) taps, so it's 4^{th} order filter.

Transposed Vs. Direct Form filter

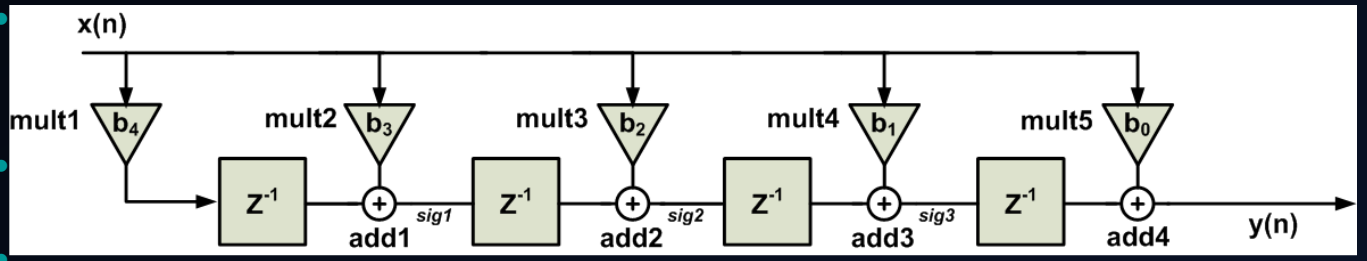


Critical Path = one mult. + N*adders where N equals FIR order

In this case:

(mult1 + add1 + add2 + add3 + add4)

$$f_s^{\max} = \frac{1}{T_{\text{mult}} + 4 \cdot T_{\text{add}}}$$



Critical Path = one mult.+ one adder whatever the order of the FIR.

(mul5 + add4) in this case.

$$f_s^{\max} = \frac{1}{T_{\text{mult}} + T_{\text{add}}}$$

- Choosing the Filter to be Designed:

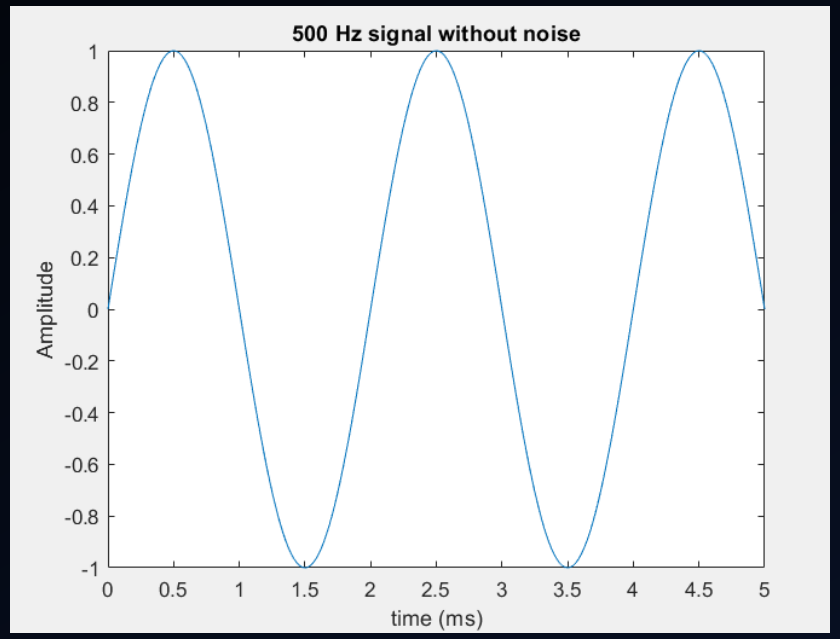
We will design **transposed** filter that follow **widowing method** in designing our FIR Filter, and our chosen window will be **hamming window**, so our design parameters will go as follows:

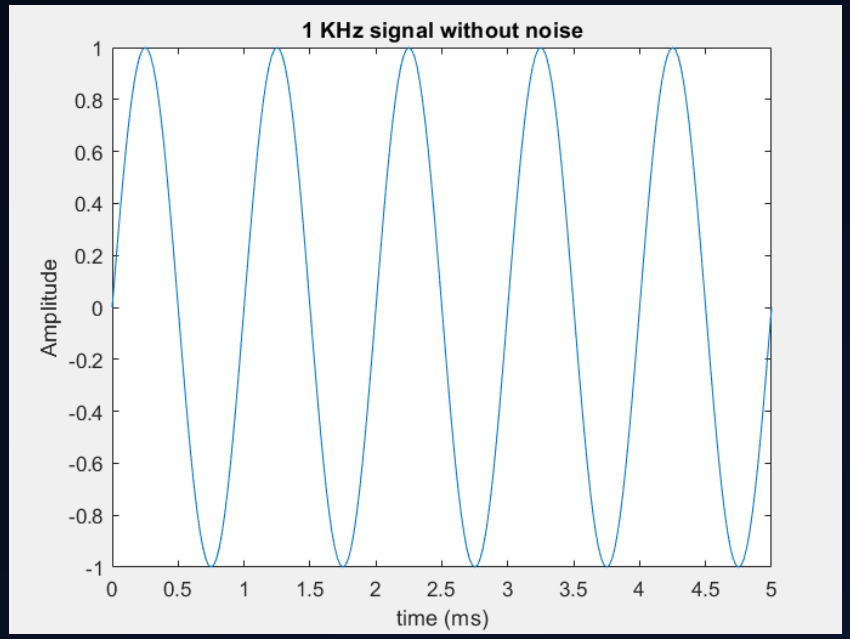
- **A 50-order (51 taps) hamming FIR LPF Filter is to be designed with a cutoff frequency of 1 KHZ for a sampling frequency of 48 KHz.**

MATLAB Golden Model

- Steps

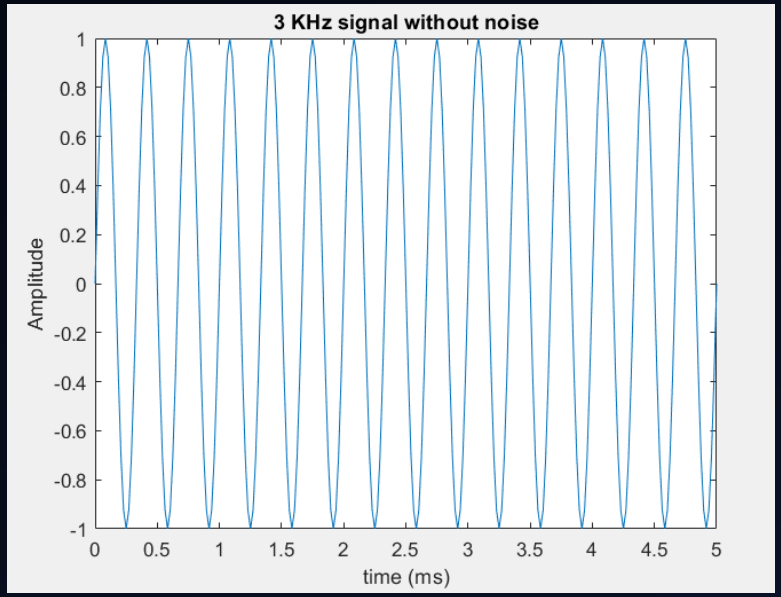
1- Generate 3 sine waves of frequencies (500,1000,3000) Hz.

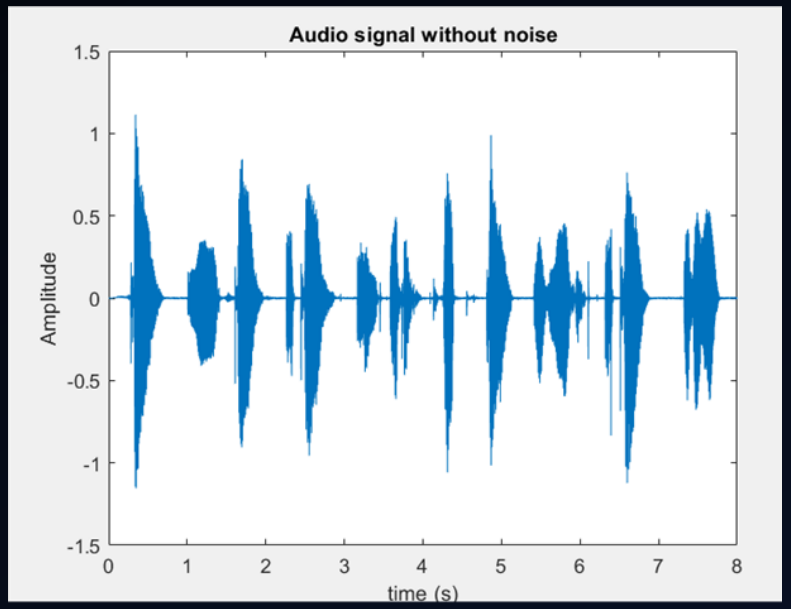




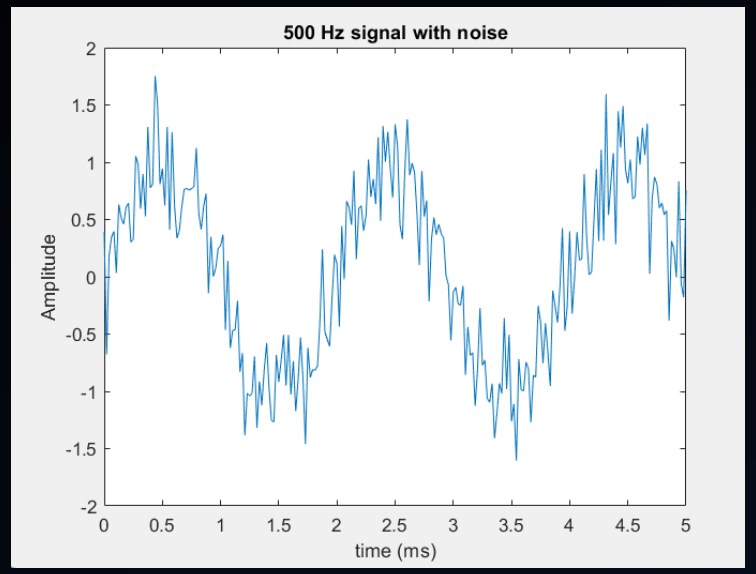
2- Read the audio recording file (can_recording.wav) and convert it into signal points.

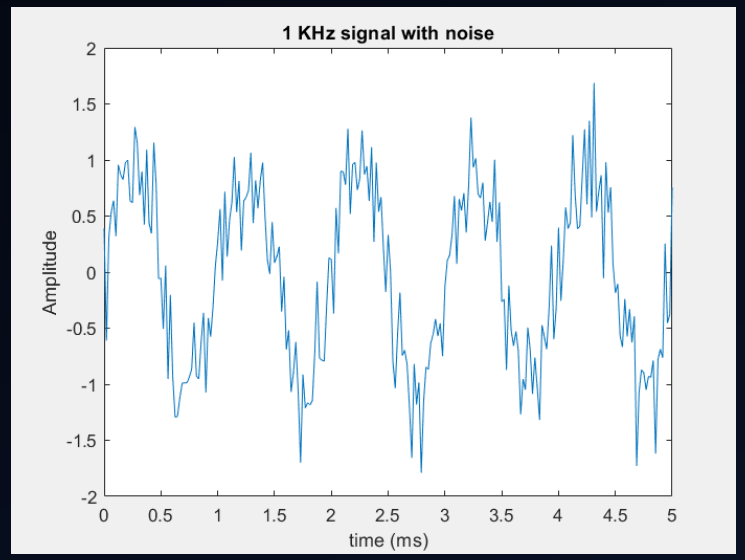
3- Choose 48 KHz sampling frequency as it’s more than the twice the highest heared audio frequency, so it’ll be suitable for both the audio & sine waves.

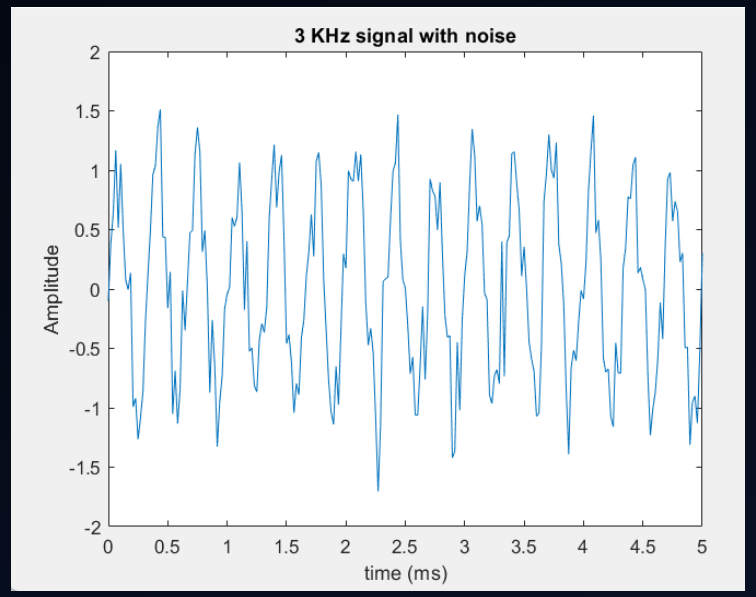


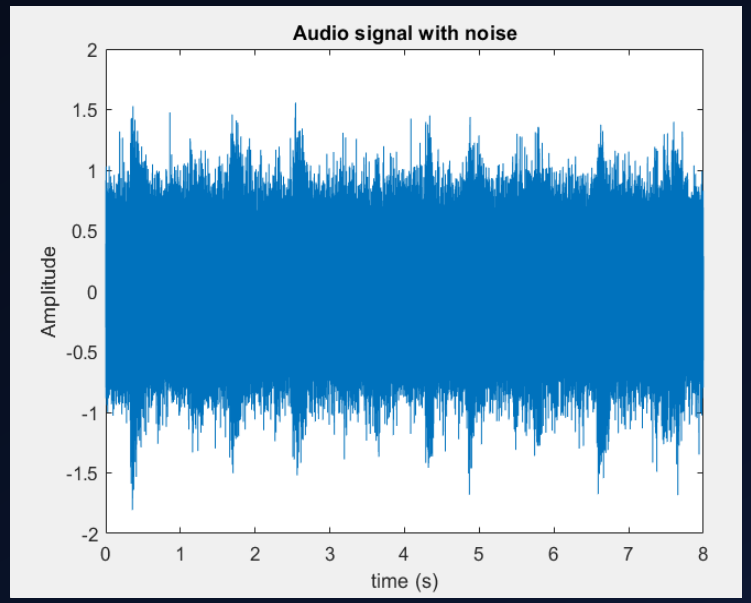


4- Add Additive white Gaussian noise (AWGN) to all the signals of mean equals zero and variance equals 0.09.



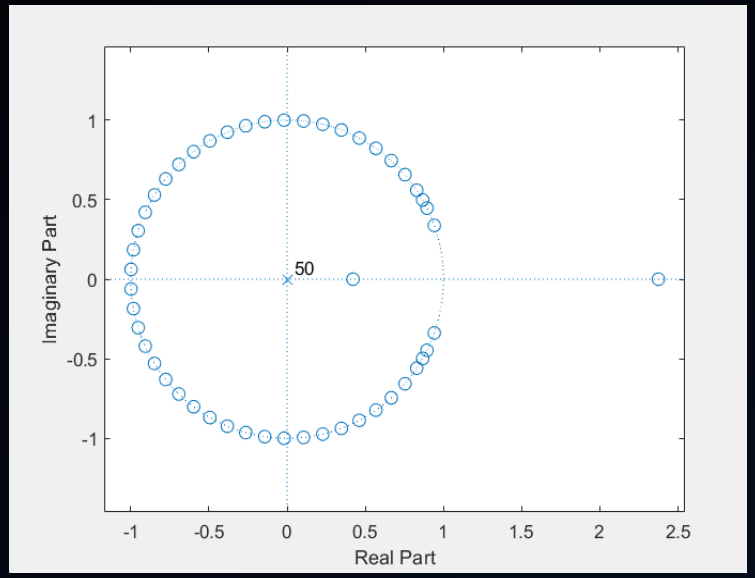


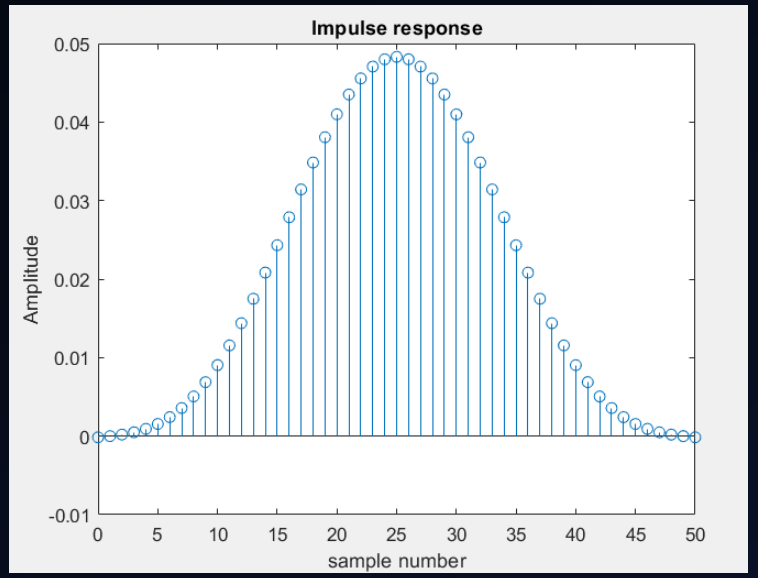


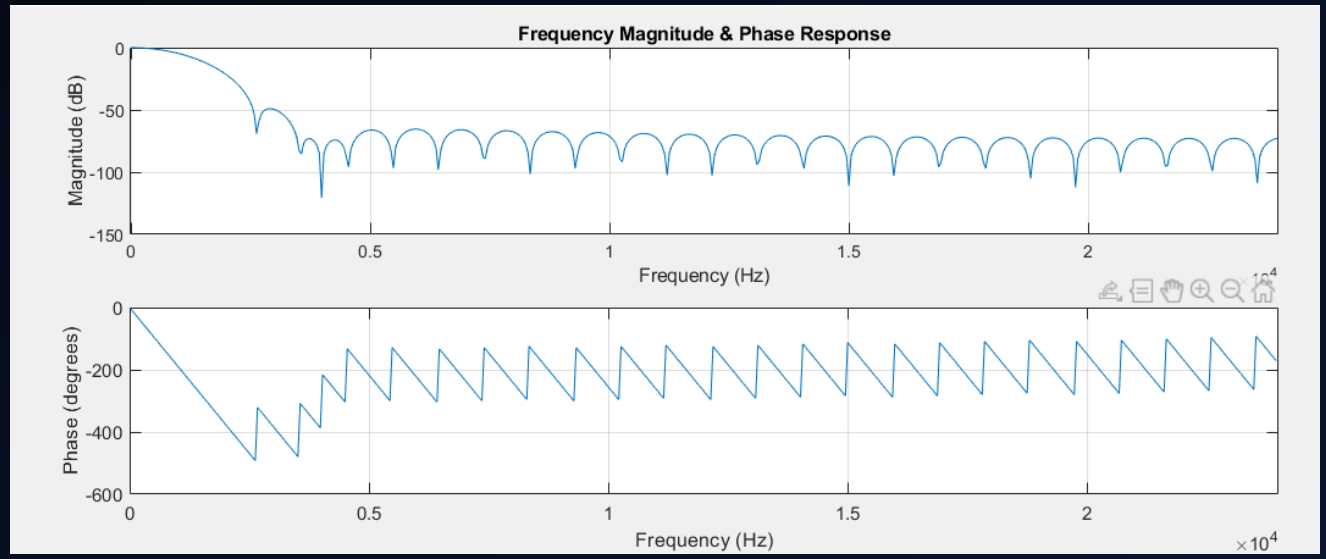


5- Write each input signals in form of fixed point in a separate txt file.

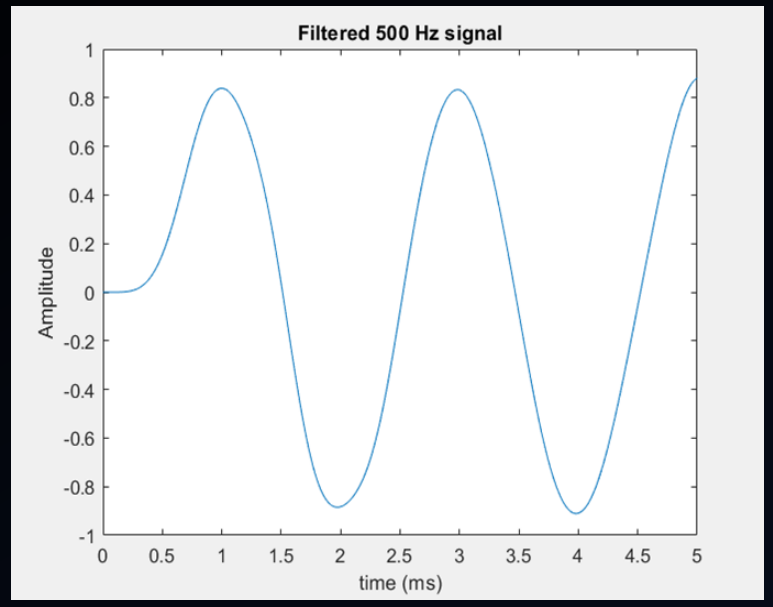
6- Generate filter coefficients according to the specs discussed before.

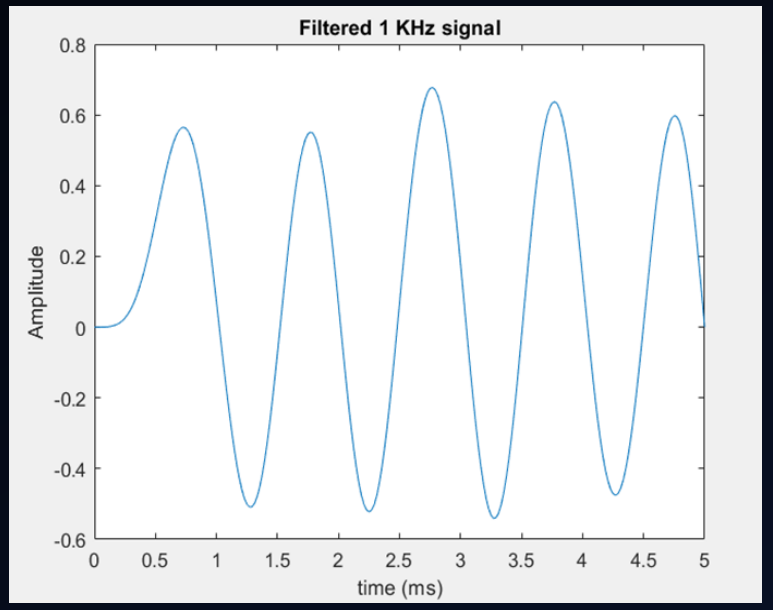


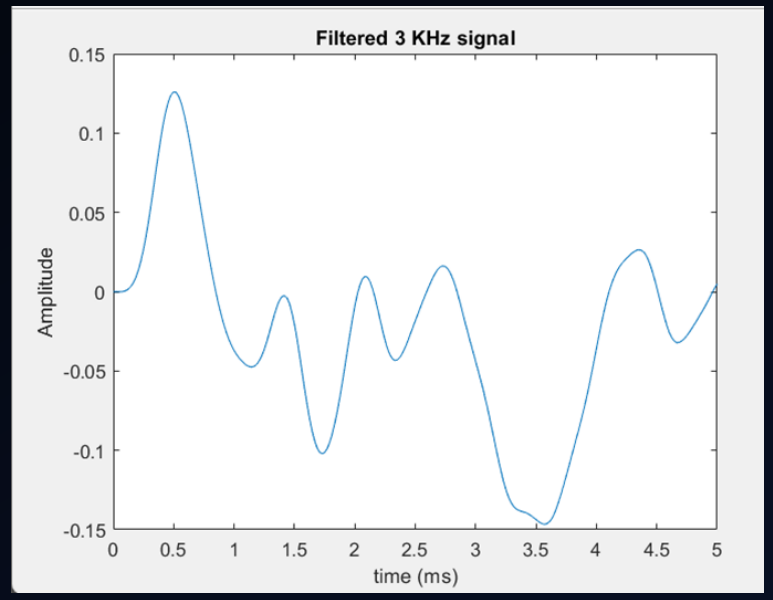


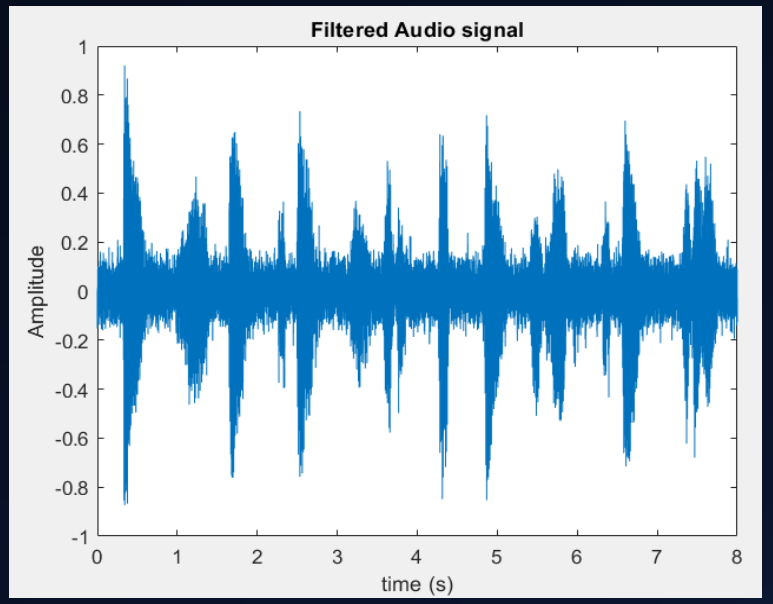


7- Apply the filter response to the 4 signals using (filter) function.









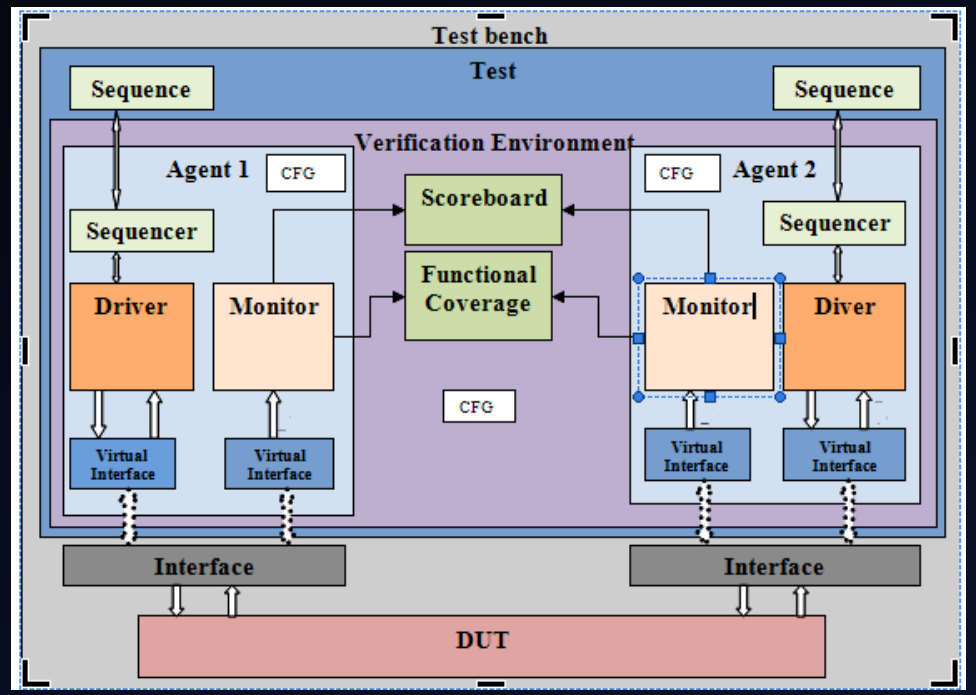
8- Write each output signal in fraction form in a separate txt file.

9- Write the audio signal in a wav file in order to hear it.

10- After running Questasim it’ll write the tested design signal in a txt file, Read it and write it in wav file, to can hear it.

FIR UVM Environment

The figure on the right shows UVM environment with 2 agents & 2 interfaces, but in our design we will need only one agent & one interface only.



- UVM objects

1- Sequence item

- Contains three main signals, noisy input, filtered output and reset.
- Takes the noisy input signal points from txt file then store it in an array.
- Also contain array for filtered signal, that will be used by Scoreboard & Coverage collector.
- Constraint the reset to be always asserted

2- Sequence

- Creates sequence item for reset assertion.
- Creates 4 sequence items, each item read an input signal (disables randomization for all of them).
- Sends the 4 sequence items to the sequencer and before each one, it sends the reset sequence item.

3- Configuration object

There’s 2 configuarion objects.

- **The First:** Stores the virtual interface and gives it to the components that’ll use it.
- **The Second:** Contains a variable used by the monitor to determine the first sequence item sent for delay purposes.

• UVM Components:

1- Interface

- Connects the DUT and the UVM environment.

2- DUT (Device Under Test)

- The desired design need to be tested.

3- Test

- Instantiates the environment
- Controls the sequences and configurations.
- Connects the coverage collector to the environment.

4- Environment

- Instantiates the Agent, Scoreboard, and Coverage Collector.
- Connects the DUT interface to the Agent.

5- Agent

- Instantiate Sequencer, Driver, and Monitor.
- Connects Sequencer & Driver with TLM.
- Drives the interface signals to the DUT via the Driver.
- Monitors the interface for outputs via the Monitor.

6- Sequencer

- Drives the sequence to the Driver.
- Sends sequence items to the Driver.

7- Driver

- Loops on the input signal array sent within the sequence item and drives each point to the virtual interface along with rest.

8- Monitor

- Collects the output filtered signal from the DUT in an array.
- Encapsulates the filtered signal array & reset in a sequence then broadcast it to the Scoreboard & Coverage Collector.

9- Coverage collector

- Receives sequence items from the monitor.
- Performs functional coverage on inputs & outputs.

10- Scoreboard

- Receives sequence items from the monitor
- Reads the filtered signal from output file from MATLAB
- Compares the filtered signal from DUT with the one MATLAB

Project phases

1- After building Questasim project named (FIR), Run (run_script.py) script.

2- The python script will open Matlab and run (Signal_Generation.m) script.

3- After Matlab finishes and you close Matlab, The python script will open FIR Questa project and run the do file (run.do).

3- The TCL commands in (run.do) will run the project, add waveforms and enable code coverage.

4- When Questa finishes simulation, it'll produce code coverage report and write filtered audio signal in a txt file.

5- After you close Questasim, The script will open Matlab again and run (Generate_audio_dut.m) script.

6- The second Matlab script will take the filtered audio txt file and convert it into heared audio wav file.

DUT Results

- Transcript output

```
# UVM_INFO FIR_sequence.sv(48) @ 4: uvm_test_top.env.ag.FIR_sequencer@@seq [SEQ_ITEM] Reset item has run successfully
# UVM_INFO FIR_sequence.sv(56) @ 4: uvm_test_top.env.ag.FIR_sequencer@@seq [FILE_READING] First file has been read successfully
# UVM_INFO FIR_scoreboard.sv(118) @ 968: uvm_test_top.env.sb [OUTPUT COMPARISON_SUCCESS] Scoreboaed compared signal no. 0 successfully with no errors
# UVM_INFO FIR_sequence.sv(62) @ 968: uvm_test_top.env.ag.FIR_sequencer@@seq [SEQ_ITEM] First item has run successfully
# UVM_INFO FIR_sequence.sv(75) @ 972: uvm_test_top.env.ag.FIR_sequencer@@seq [FILE_READING] Second file has been read successfully
# UVM_INFO FIR_scoreboard.sv(118) @ 1936: uvm_test_top.env.sb [OUTPUT COMPARISON_SUCCESS] Scoreboaed compared signal no. 1 successfully with no errors
# UVM_INFO FIR_sequence.sv(80) @ 1936: uvm_test_top.env.ag.FIR_sequencer@@seq [SEQ_ITEM] Second item has run successfully
# UVM_INFO FIR_sequence.sv(94) @ 1940: uvm_test_top.env.ag.FIR_sequencer@@seq [FILE_READING] Third file has been read successfully
# UVM_INFO FIR_scoreboard.sv(118) @ 2904: uvm_test_top.env.sb [OUTPUT COMPARISON_SUCCESS] Scoreboaed compared signal no. 2 successfully with no errors
# UVM_INFO FIR_sequence.sv(99) @ 2904: uvm_test_top.env.ag.FIR_sequencer@@seq [SEQ_ITEM] Third item has run successfully
# UVM_INFO FIR_sequence.sv(111) @ 2908: uvm_test_top.env.ag.FIR_sequencer@@seq [FILE_READING] audio file has been read successfully
# UVM_INFO FIR_scoreboard.sv(118) @ 1538908: uvm_test_top.env.sb [OUTPUT COMPARISON_SUCCESS] Scoreboaed compared signal no. 3 successfully with no errors
# UVM_INFO FIR_sequence.sv(116) @ 1538908: uvm_test_top.env.ag.FIR_sequencer@@seq [SEQ_ITEM] audio item has run successfully
# UVM_INFO FIR_test.sv(57) @ 1538908: uvm_test_top [RUN_PHASE] Ending run_phase
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 1538908: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO :   21
# UVM_WARNING :    0
# UVM_ERROR :    0
# UVM_FATAL :    0
# ** Report counts by id
# [BUILD_PHASE]     2
# [FILE_READING]     4
# [OUTPUT COMPARISON_SUCCESS]     4
# [Questa UVM]     2
# [RNTST]     1
# [RUN_PHASE]     2
# [SEQ_ITEM]     5
# [TEST_DONE]     1
```

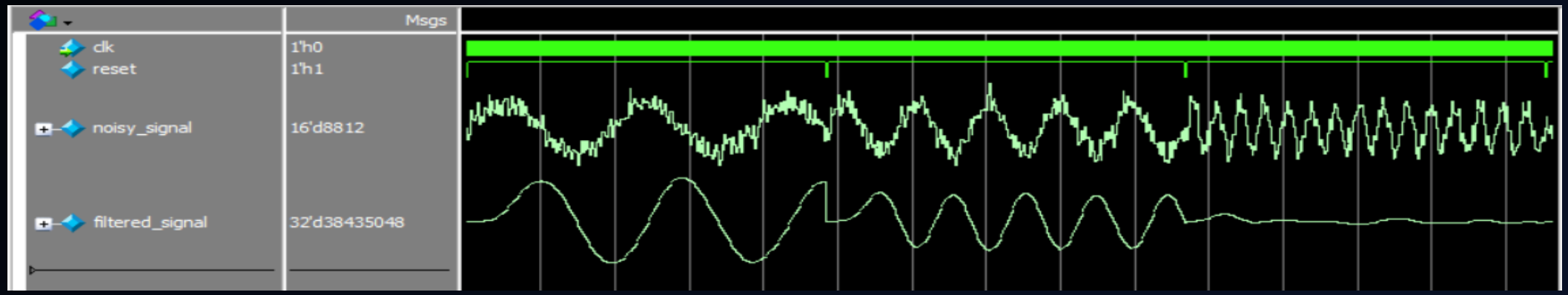
- Functional coverage

/Top_module_sv_unit/FIR_coverage	92.44%			
TYPE c1	92.44%	100	92.44%	✓
CVP c1::c_reset	100.00%	100	100.00...	✓
CVP c1::c_noisy_signal	80.46%	100	80.46%	✓
CVP c1::c_filtered_signal	96.87%	100	96.87%	✓

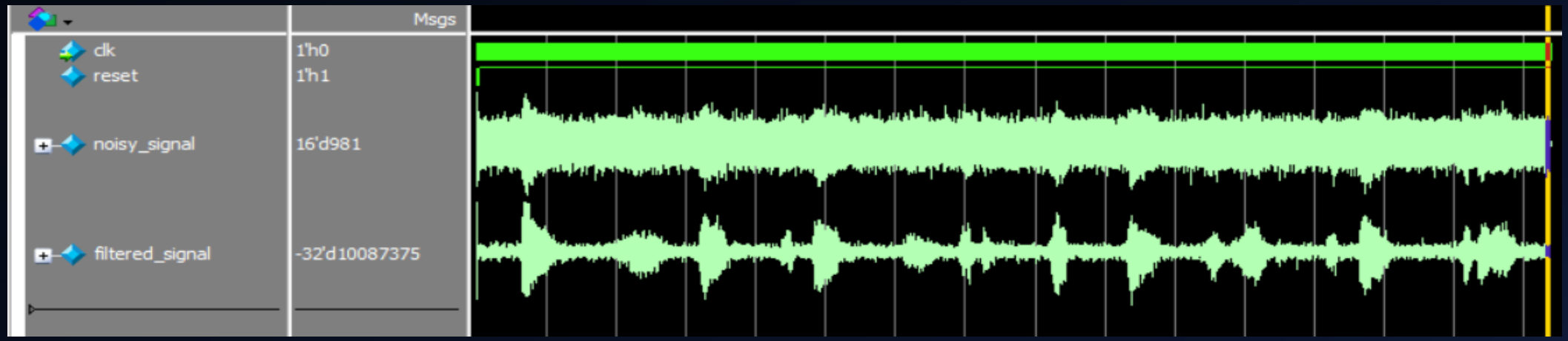
- Code Coverage

```
Total Coverage By Instance (filtered view): 95.29%
```

- Sine waveforms



- Audio waveform



Thank You

GITHUB: MWAEL2002

LINKEDIN: MOHAMMAD WAEL